

Exploring Hybrid DFT– Quantum Embedding Simulations of Molecules for HPCQC

Presented By – Dr. Namrataa Kkommineni,
Tejjan Arora,
Soham Phulare,
Shreyas Kadam,

Shah & Anchor Kutchhi Engineering College
Guided By - Samrit Kumar Maity, Scientist E , HPC
technologies, CDAC-Pune.



Outline of the presentation



DFT Basics



DFT-Based Quantum Embedding



Quantum Solver Basics



Results & Discussion



Students Progress



Conclusion

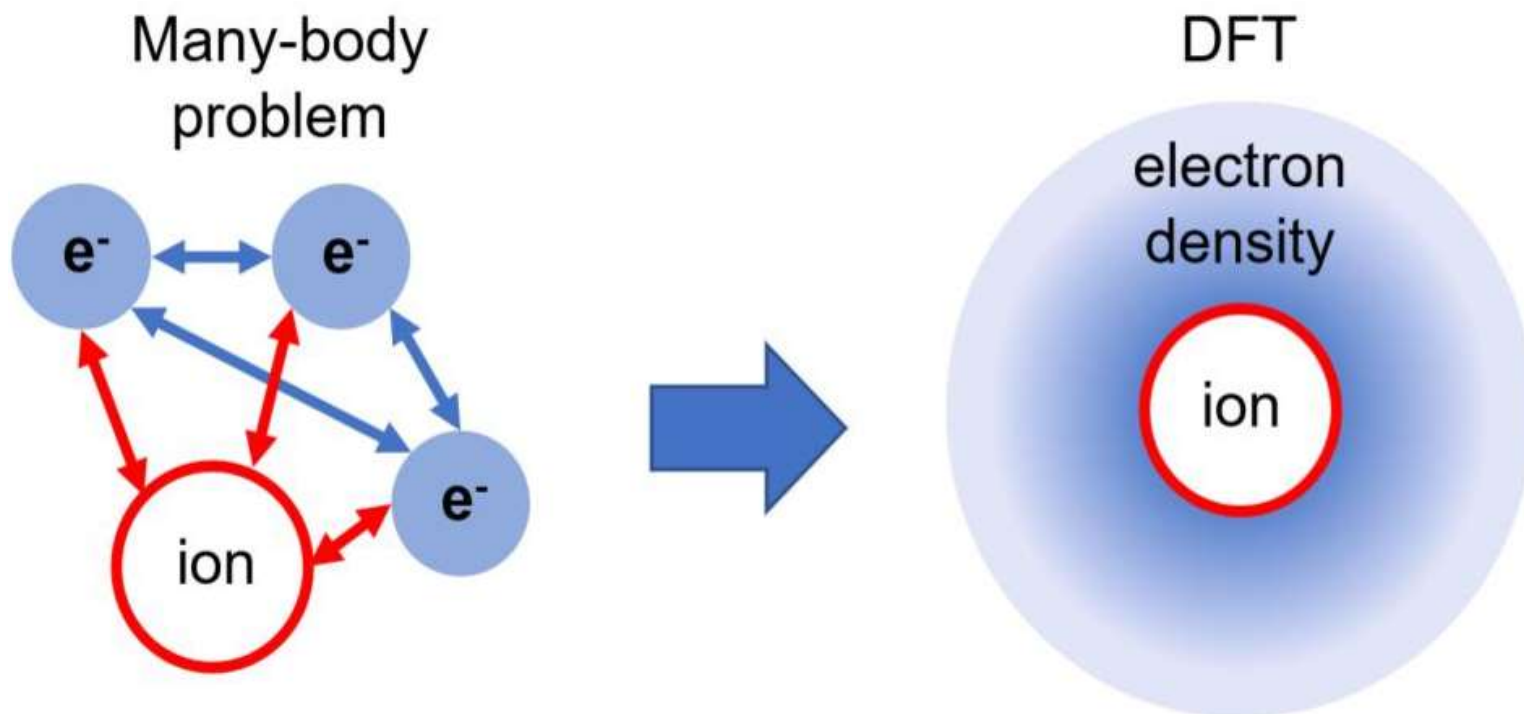


Future Plans

★ What is DFT?

Density Functional theory

DFT is a **quantum-mechanical method** that calculates the **electronic structure** of molecules and materials. It uses the (a function of 3 spatial variables) **-electron density** instead of many electron wavefunctions, making it computationally efficient. By solving electrons moving in an **effective potential**, DFT predicts **ground-state properties** with good accuracy and practical cost.



✦ Why do we need DFT ?

❓ **Broad Applicability of DFT**

- Used to study molecules, solids, surfaces, and reactions
- Applied in chemistry, physics, materials science, and biology

🌐 **Essential Tool for Domain Scientists**

- Enables chemists, physicists, and engineers to predict electronic and structural properties
- Bridges experiments and theory with reliable, first-principles insights

❓ **Pillar of Modern Scientific Simulation**

- Balances accuracy and efficiency for large-scale simulations
- Backbone of multiscale, hybrid, and emerging quantum-classical methods

Limitations of DFT and way ahead

Computational Limitations of DFT

- Accuracy relies on approximate exchange–correlation functionals
- Cannot fully capture strong correlation, excited states, or bond-breaking processes
- Classical DFT often underestimates multi-reference effects

Scaling and Resource Bottlenecks

- Computational cost grows rapidly with system size and basis set
- High-accuracy simulations of large molecules require significant HPC resources
- Limits predictive modeling for complex or strongly correlated materials

Pathway to HPC–Quantum Computing (HPCQC)

- Quantum algorithms naturally handle strong correlation in the active region
- Hybrid HPC–QC workflows combine classical DFT environment with quantum solvers
- Extend simulations to regimes beyond the reach of classical methods alone



What is Quantum Embedding

❓ **Concept: Embedding Framework**

- Decompose complex electronic systems into smaller, manageable subsystems
- Enable efficient and accurate simulations of large molecules

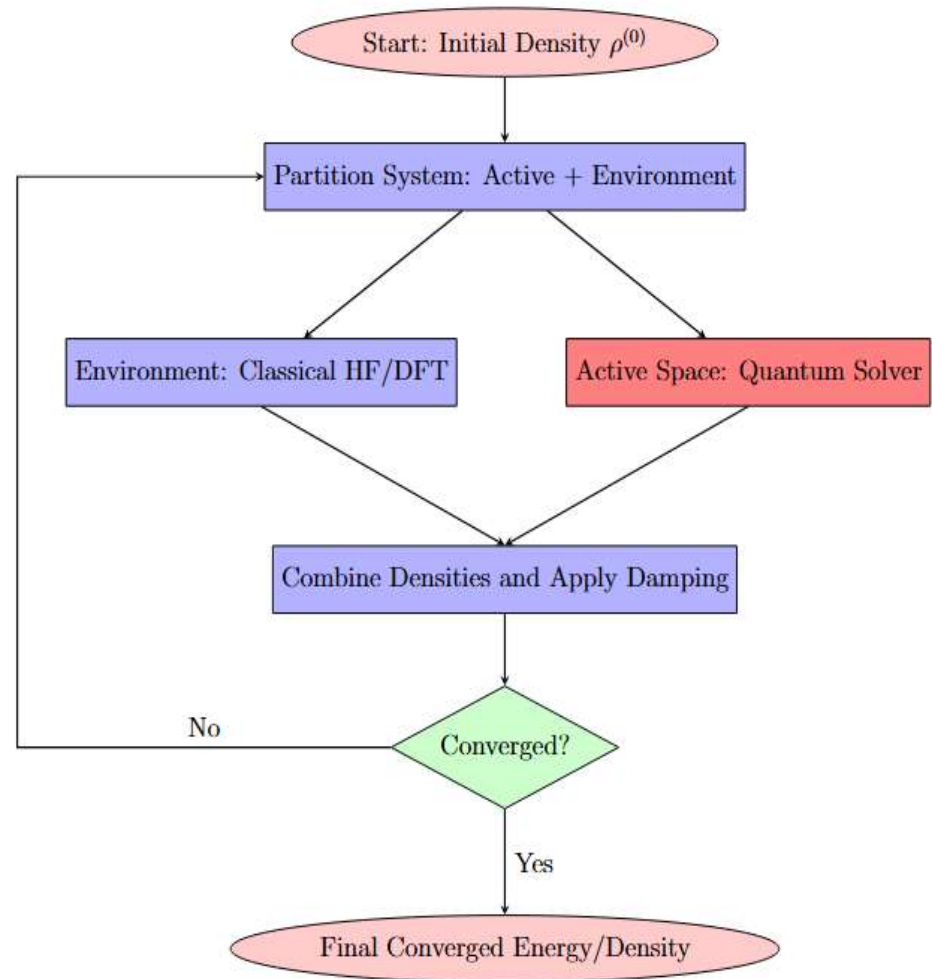
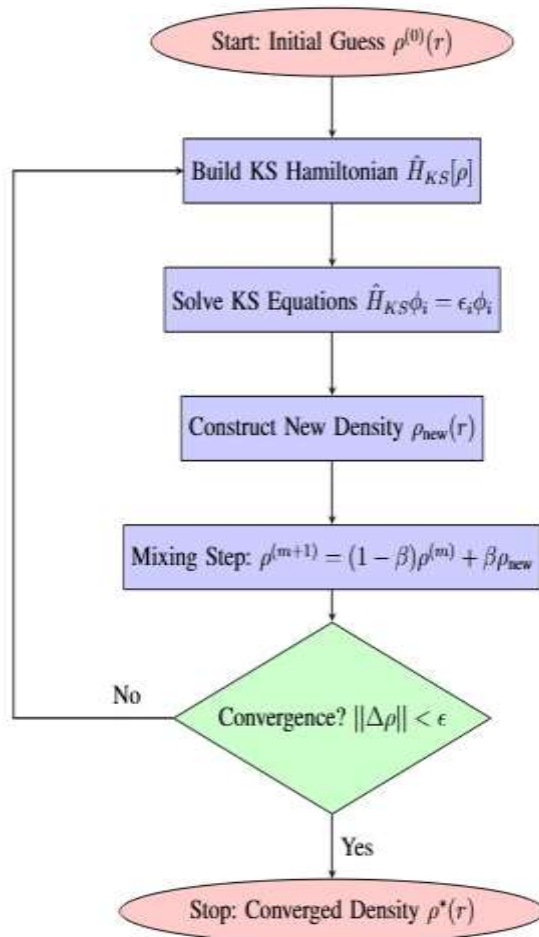
⚙️ **Embedding Strategy**

- Treat the chemically active region with high-accuracy quantum methods (QCA)
- Describe the surrounding environment using HF/DFT for computational efficiency

🚀 **Quantum Advantage**

- Capture strong correlation and multi-reference effects beyond HF/DFT
- Reduce quantum resource requirements by restricting quantum treatment AS.

✦ DFT Vs Quantum Embedding (QDFT)



★ Quantum Solver (VQE)

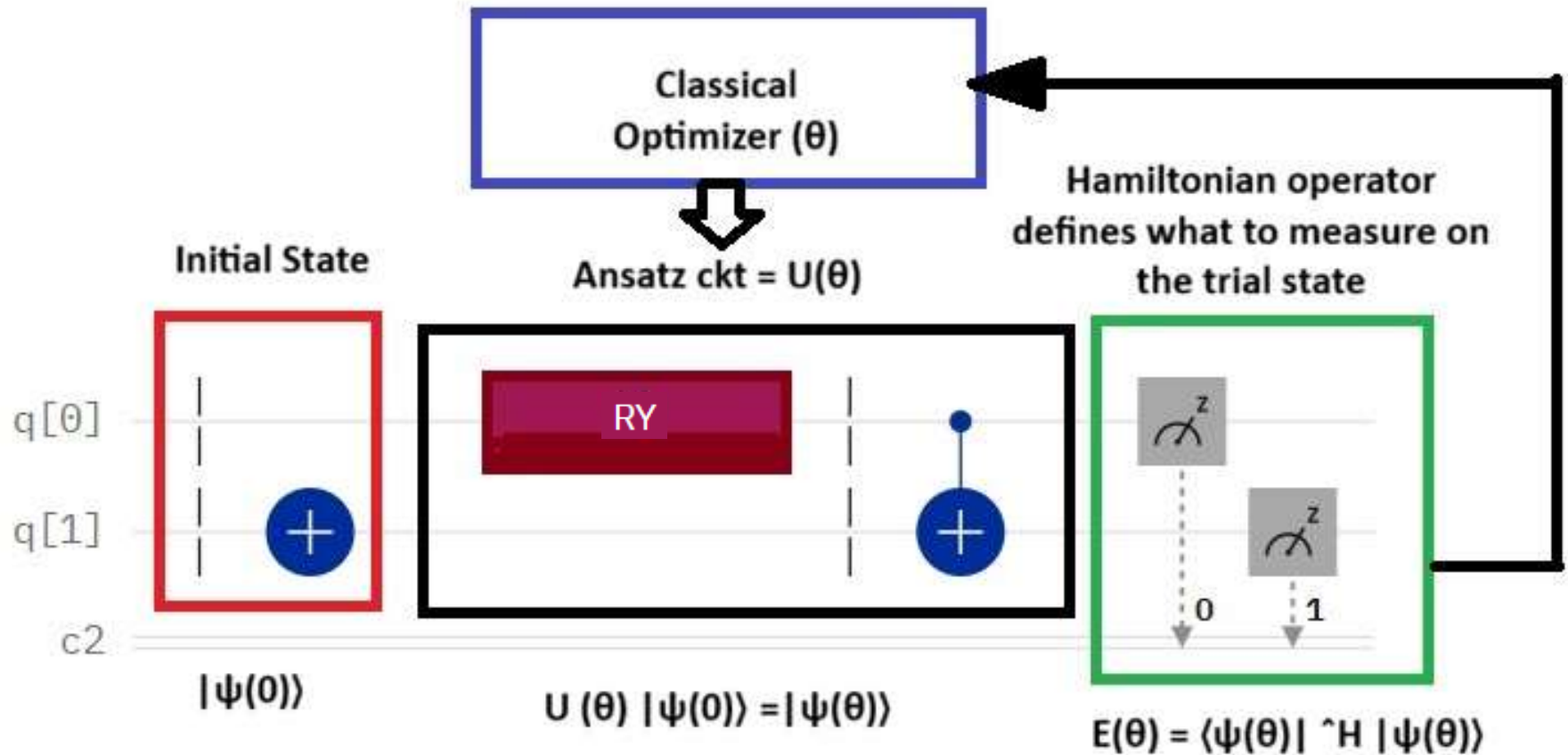
Variational Principle

The Variational Principle is a fundamental concept underlying the Variational Quantum Eigensolver (VQE). It states that for any normalized trial wavefunction $|\psi(\theta)\rangle$, the expectation value of the Hamiltonian is always greater than or equal to the true ground-state energy E_0 .

Steps for VQE

- Start with initial **HF state** $|\psi(0)\rangle$.
- The **ansatz circuit** $U(\theta)$ $|\psi(0)\rangle = |\psi(\theta)\rangle$, trial state prepared on a quantum computer.
- The **classical optimizer** adjusts θ to minimize $E(\theta)$.
- Repeat till either max number of iterations or energy threshold reached .
- **NOTE:** Regardless of the ansatz choice, the measured energy cannot go below E_0
- **NOTE:** A more expressive ansatz allows $E(\theta)$ to approach E_0 more closely.

★ VQE (Ansatz-Optimizer-Hamiltonian)



System Configuration

Compute Node

- Intel Xeon Gold 6240R processors
- 48 physical cores (2 × 24 cores)
- 2.40 GHz base frequency (up to 4.0 GHz turbo)

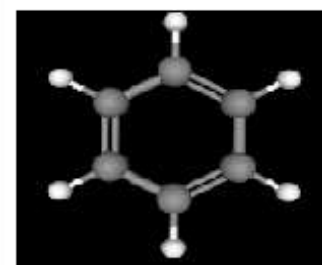
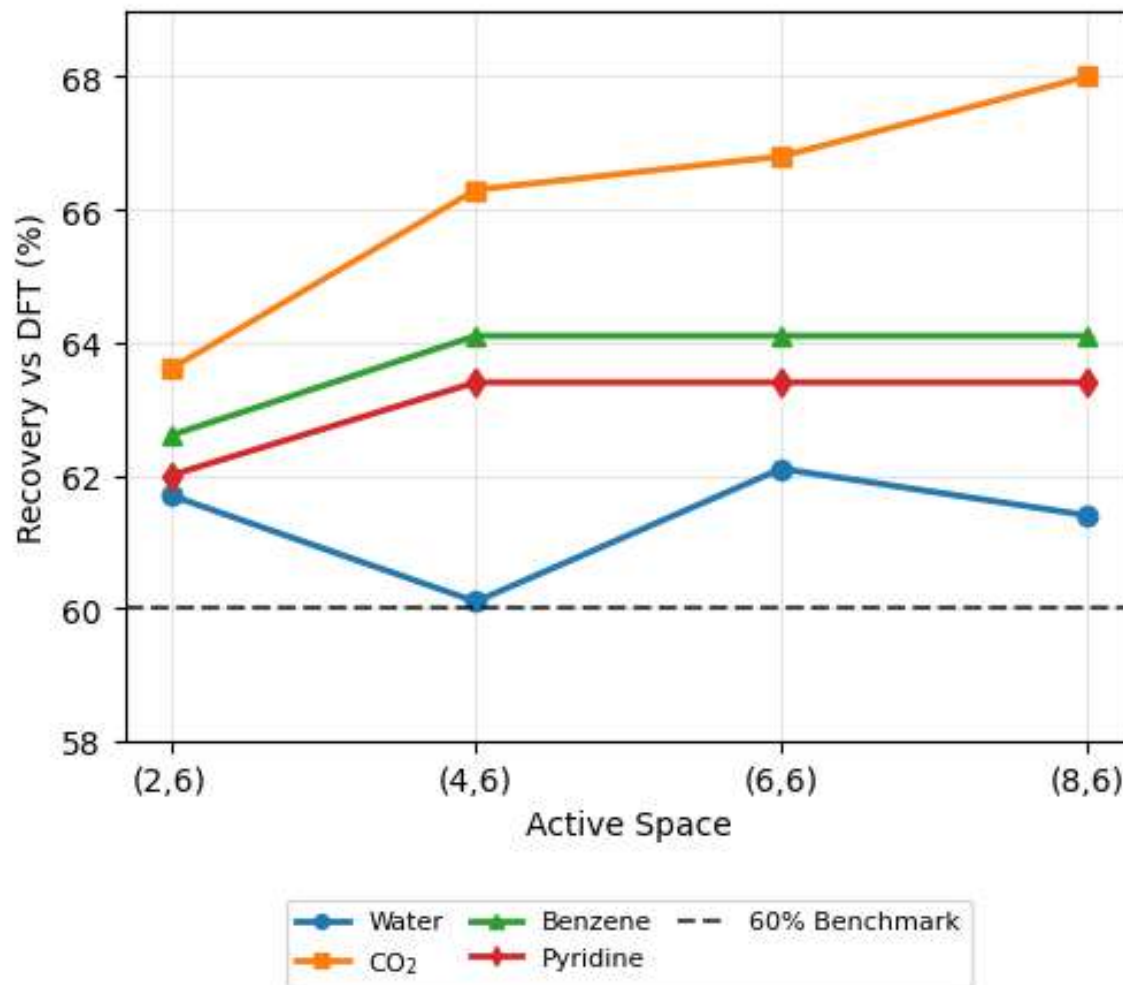
Memory Architecture

- 192 GB total RAM per node
- ~187 GB usable memory

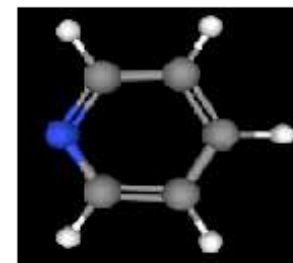
Execution Model

- Single-node, shared-memory execution
- Full-node allocation (48 cores utilized)

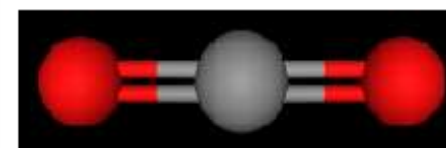
Correlation Recovery Results



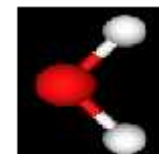
Benzene



Pyridine



Carbon dioxide



Water

Energy Deviation Results

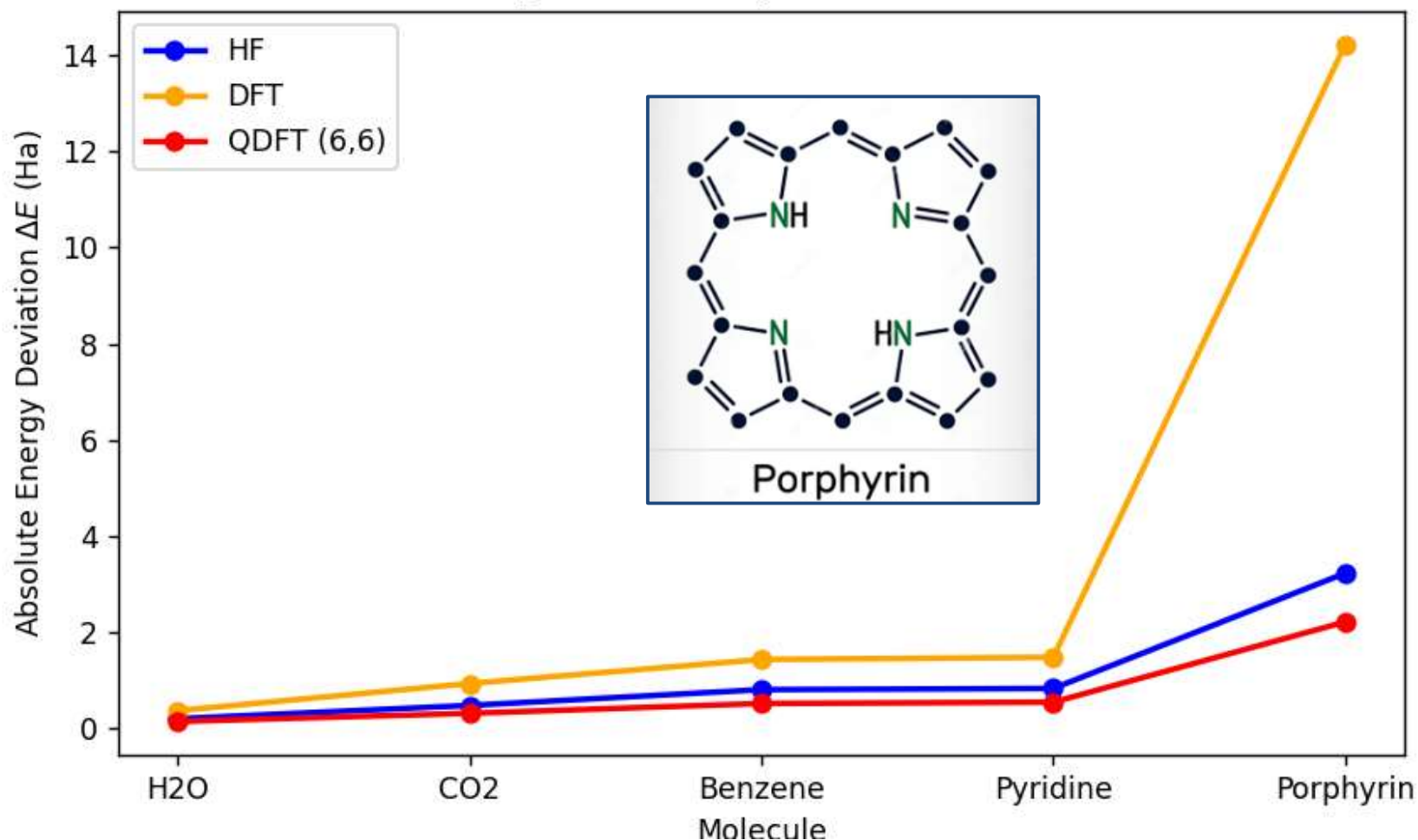
Table 6: Absolute energy deviation $\Delta E = |E_{\text{method}} - E_{\text{CCSD}}|$ (Ha) for HF, DFT, and QDFT (6,6) across selected molecules.

Molecule	ΔE HF (Ha)	ΔE DFT (Ha)	ΔE QDFT (6,6) (Ha)
H ₂ O	0.19619	0.36315	0.13773
CO ₂	0.47183	0.92859	0.30803
Benzene	0.80111	1.42824	0.51255
Pyridine	0.82569	1.47553	0.54011
Porphyrin	3.22500	14.22354	2.21114

Disclaimer: QDFT results are obtained via noiseless quantum simulations in Qiskit on classical hardware.

Energy Deviation Results

Energetic Accuracy Relative to CCSD



Disclaimer: QDFT results are obtained via noiseless quantum simulations in Qiskit on classical hardware.

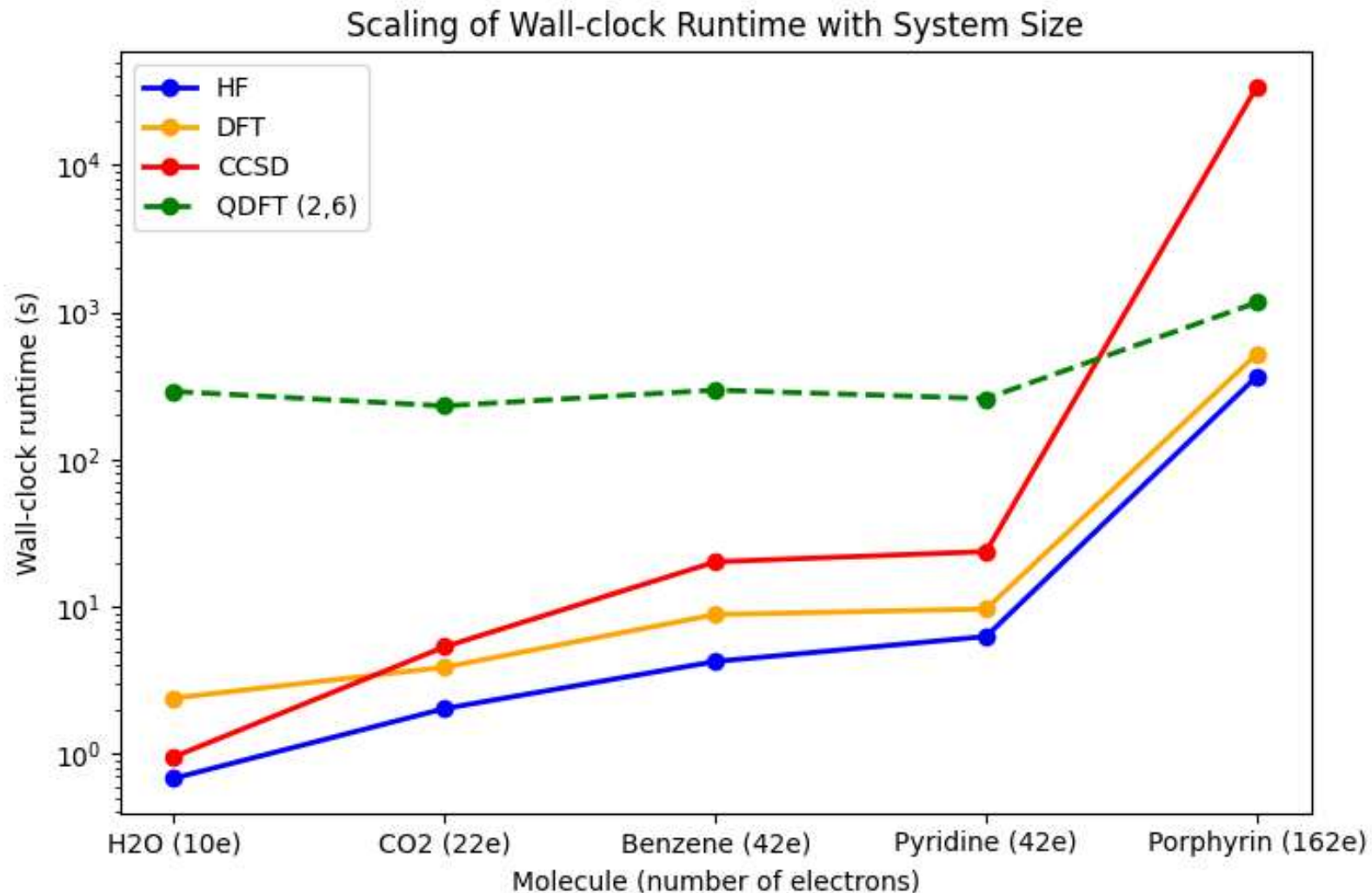
Run Time table

Table 5: Wall-clock runtimes [s] for different electronic structure methods across selected molecules. The total number of electrons is also listed.

Molecule	Electrons	HF [s]	DFT [s]	CCSD [s]	QDFT (2,6) [s]	QDFT (6,6) [s]
H ₂ O	10	0.682	2.393	0.953	289.63	1954.93
CO ₂	22	2.030	3.876	5.336	230.91	1969.30
Benzene	42	4.242	8.837	20.149	295.90	1976.16
Pyridine	42	6.269	9.640	23.663	259.16	1984.43
Porphyrin	162	366.928	521.100	34092.942	1163.99	3192.17

Disclaimer: QDFT results are obtained via noiseless quantum simulations in Qiskit on classical hardware.

Run Time Scaling



Disclaimer: QDFT results are obtained via noiseless quantum simulations in Qiskit on classical hardware.

★ QDFT- Time Scaling

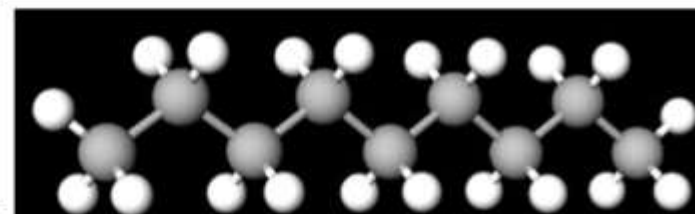
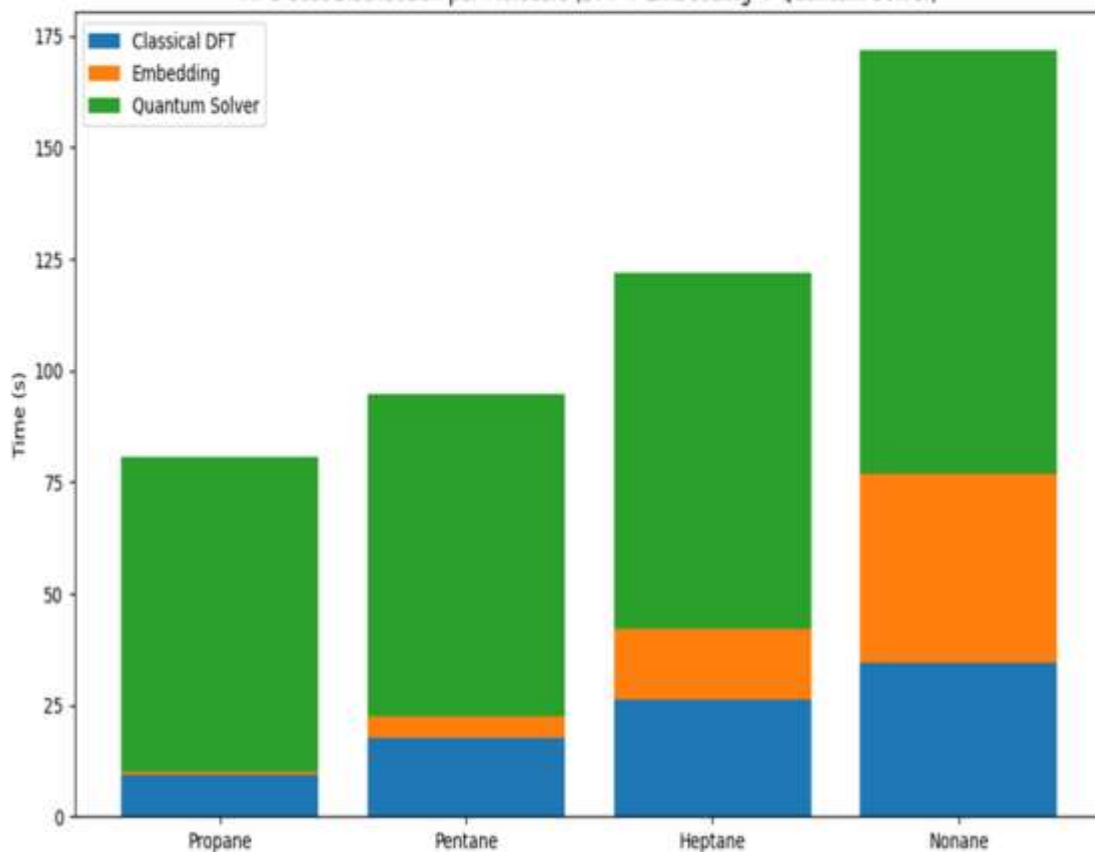
Table 5: Runtime and memory for QDFT CAS(2e,4o)

Molecule	Chemical Formula	DFT Time (s)	Embedding Time (s)	VQE Time (s)	Peak RSS (MB)
Propane	C ₃ H ₈	9.49	0.70	70.29	575
Pentane	C ₅ H ₁₂	17.71	4.67	72.36	760
Heptane	C ₇ H ₁₆	26.33	15.89	79.73	1344
Nonane	C ₉ H ₂₀	34.52	42.30	94.93	2357

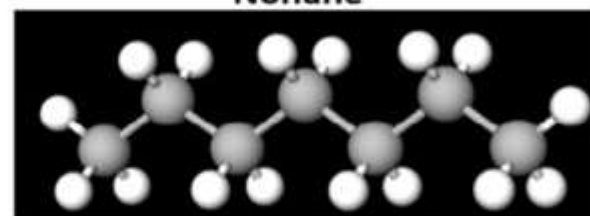


QDFT- Time Scaling

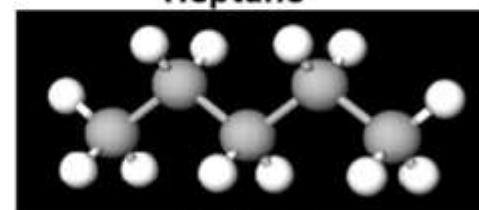
HPC Cost Distribution per Molecule (DFT + Embedding + Quantum Solver)



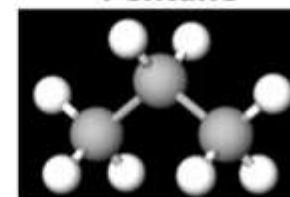
Nonane



Heptane



Pentane



Propane

Runtime Result & Discussion

Runtime Scales with Electrons

- All methods show increasing runtime as the number of electrons grows

Classical Methods Remain Efficient

- HF and DFT are fast for small to medium electron counts
- CCSD runtime grows faster but is still manageable

QDFT Runtime Sensitive to System Size

- Embedding becomes costly as electron count increases
- Runtime grows significantly compared to classical methods

Implications for Workflow Design

- Small systems → classical methods sufficient
- Large systems → require optimized QDFT, qubit reduction, and hybrid strategies

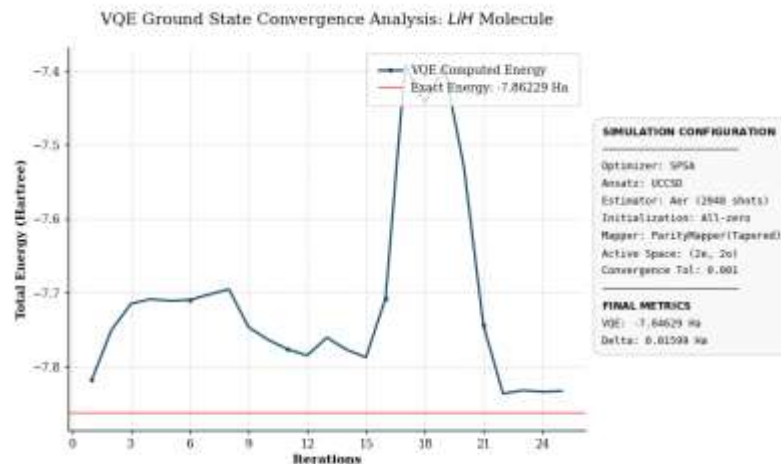
Students Progress

WORK COMPLETED BY
SOHAM PHULARE

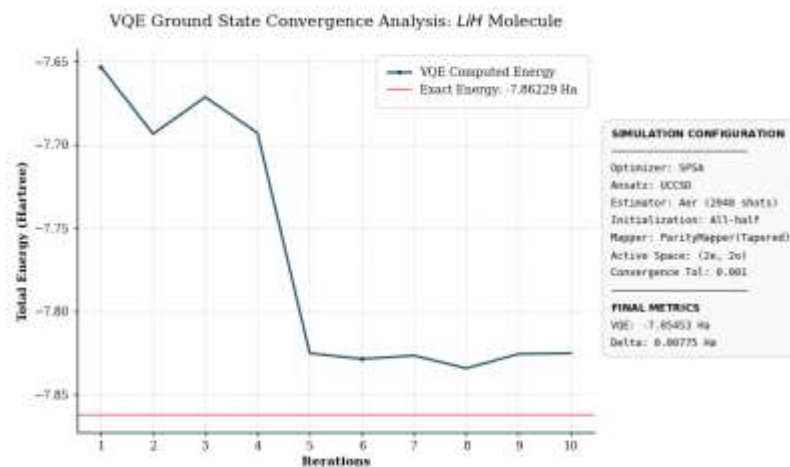
LiH | Aer Estimator | Effect of initialization

Objective: Benchmarking effect of initialization

- **Comparative Performance:** Evaluated Zero vs. Half vs. One vs Random on LiH.
- **Initialization Strategy:** Validated that **Hartree-Fock (HF)** initialization reduces iteration-to-convergence count compared to random initial states, optimizing "Warm Start" performance.
- **Conclusion:** Zero initialization causes the energy to not be damped; meanwhile small initialization (half or one) causes the damping of energy to be better



Time taken: 21.104s

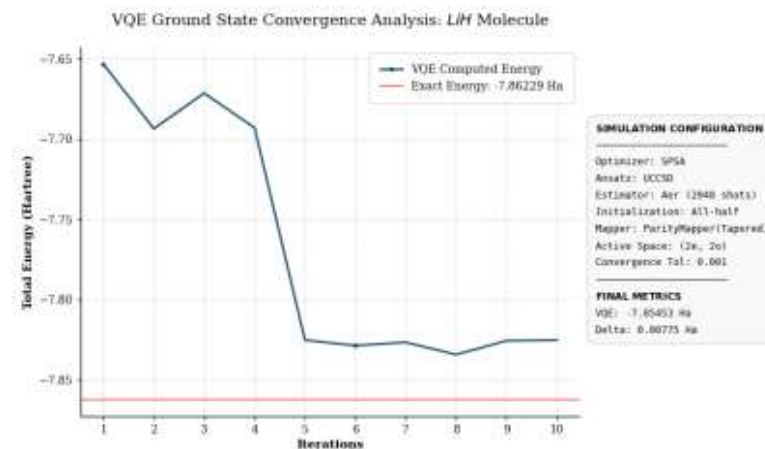


Time taken: 17.869s

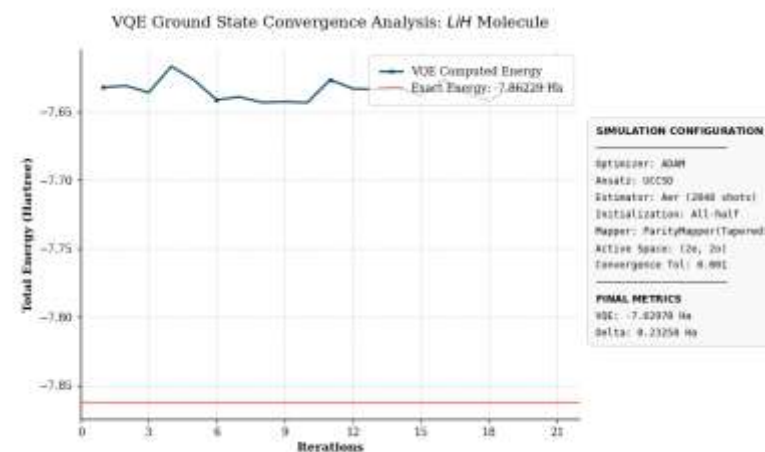
LiH | Aer Estimator | Effect of optimizer

Objective: Benchmarking effect of optimizer

- **Comparative Performance:** Evaluated SPSA vs ADAM vs Gradient Descent on LiH.
- **Optimization Strategy:** Validated that **SPSA** effectively handles stochastic noise, requiring fewer measurements per iteration than standard gradient-based approaches.
- **Conclusion:** SPSA provides significantly better convergence behavior than the ADAM optimizer, successfully minimizing the energy toward the exact ground state.



Time taken: 17.869s

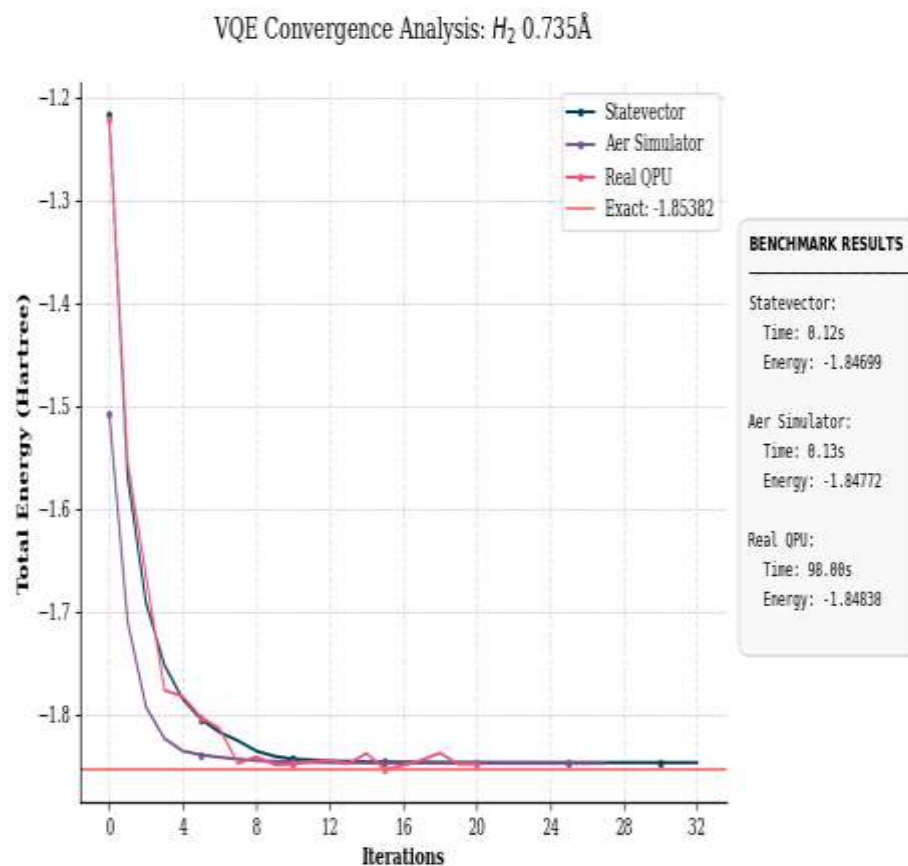


Time taken: 22.654s

H2 | SV vs Aer vs QPU Run | Results

Objective: Scaling Analysis & Bottleneck Diagnostics

- **Diagnosis:** Identified a **Circuit Depth vs. Coherence Time** bottleneck. High gate-count for UCCSD Ansatz exceeded the hardware execution window.
- **Hardware Success:** Achieved ground state energy for H_2 on QPU, successfully verified the execution pipeline.
- **Time Results:** The time taken for QPU also includes the queue time on the IBM Quantum cloud, transpilation overhead, and network latency.
- **The Scaling Bottleneck:** H_2O (2e, 6o) hardware attempts resulted in "Time Limit Exceeded" errors.



Note: Exact energy calculated using FCI

WORK COMPLETED BY

TEJJAN ARORA

Exchange Functional PBE Convergence Test

Table 1: DFT Embedding with PBE Exchange: Energy Analysis

Molecule	Active Space	Reference DFT Energy (Ha)	Final Ground-State Energy (Ha)
H ₂	(2e, 2o)	−1.161905	−1.182605
H ₂ O	(2e, 6o)	−76.320335	−76.341891
H ₂ O	(8e, 6o)	−76.320335	−76.353242
O ₂	(2e, 6o)	−150.174613	−150.136870
O ₂	(8e, 6o)	−150.174613	−150.300681

Table 2: DFT Embedding with PBE Exchange: Convergence Analysis

Molecule	Active Space	Converged at Iteration	Energy Tolerance (Ha)	Wall Time(sec)
H ₂	(2e, 2o)	9	$7.669e-08$	61
H ₂ O	(2e, 6o)	9	$2.321e-07$	1081
H ₂ O	(8e, 6o)	9	$2.54e-07$	5,516
O ₂	(2e, 6o)	4	$3.599e-07$	141
O ₂	(8e, 6o)	27	$5.873e-07$	9217

Exchange Functional B3LYP Convergence Test

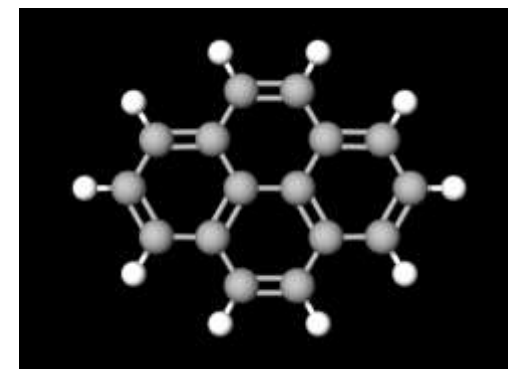
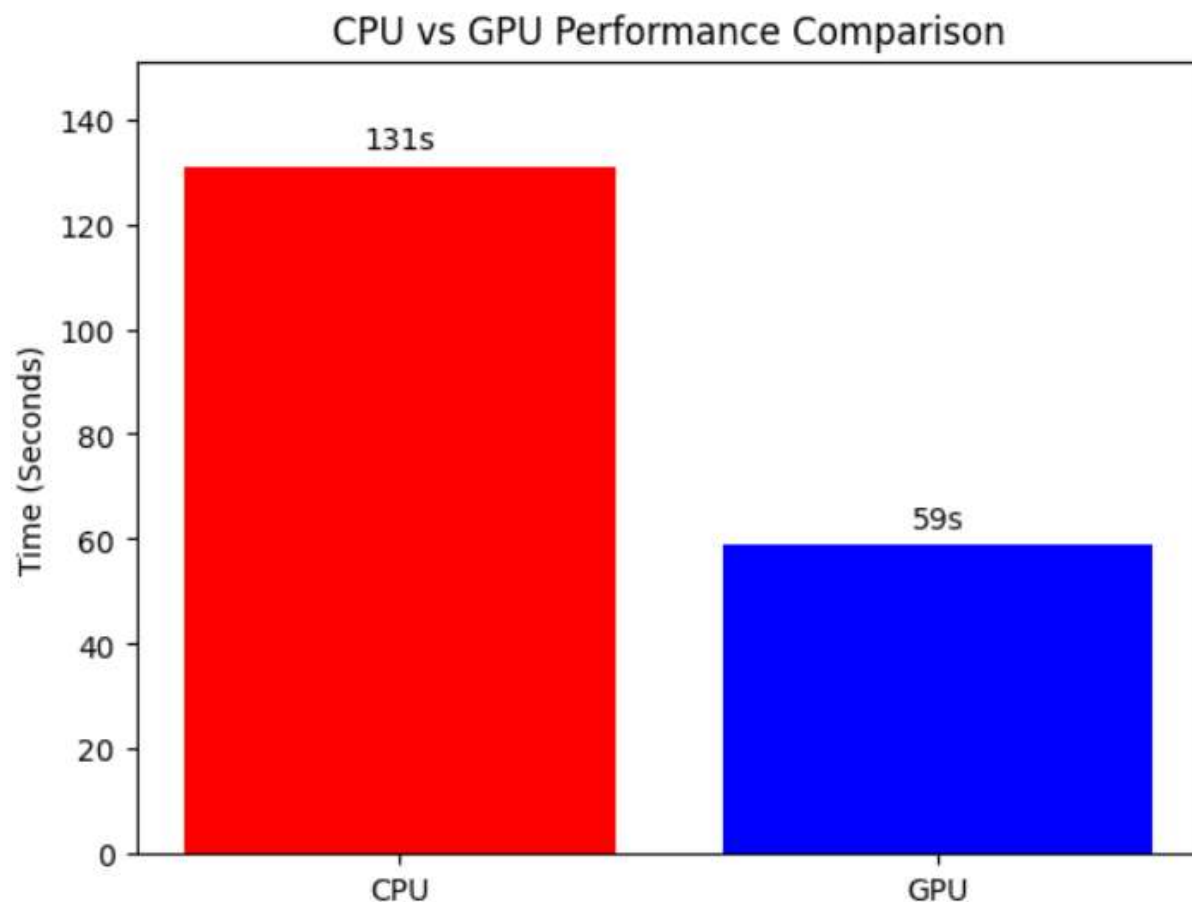
Table 3: B3LYP Exchange: Energy Analysis ($\alpha_{\text{init}} = 0.6$)

Molecule	Active Space	Reference DFT Energy (Ha)	Final Ground-State Energy (Ha)
H ₂	(2e, 2o)	-1.175477	-1.190061
H ₂ O	(2e, 6o)	-76.407023	-76.425277
H ₂ O	(8e, 6o)	-76.407023	-76.431931
O ₂	(2e, 6o)	-150.312682	-150.220130
O ₂	(8e, 6o)	-150.312682	-150.353251

Table 4: B3LYP Exchange: Convergence Analysis ($\alpha_{\text{init}} = 0.6$)

Molecule	Active Space	Converged at Iteration	Energy Tolerance (Ha)	Wall Time(sec)
H ₂	(2e, 2o)	9	$6.683e-07$	61
H ₂ O	(2e, 6o)	9	$9.587e-07$	1115
H ₂ O	(8e, 6o)	10	$7.01e-07$	6,052
O ₂	(2e, 6o)	3	$7.153e-07$	123
O ₂	(8e, 6o)	14	$5.23e-07$	4,727

CPU vs GPU Comparison Plot (Pyrene Molecule)



Pyrene ($C_{16}H_{10}$)

This comparison is justified by the fact that the GPU and CPU calculations produced identical energies (i.e. -615.760309 Hartree).

Core libraries such as **PySCF** and **Qiskit** use **C/Rust backends designed for CPU RAM (NumPy arrays)**.

GPU arrays (**CuPy**) store data in **VRAM**, which these backends cannot directly access.

Python safety layers intercept this mismatch before a crash:

- **PySCF**: CuPy arrays trigger validation errors (e.g., *AssertionError* during density matrix operations).
- **Qiskit**: CuPy blocks automatic device-to-host conversions, causing a **TypeError**.

Dynamic Namespace Injection

- Used Python's **sys.modules** to intercept all **NumPy imports**.
- Redirected them to the **CuPy GPU library** during runtime initialization.
- This forced the program to **use GPU arrays instead of CPU arrays** without modifying the original source code.

API Differences Between NumPy and CuPy

- **Missing administrative modules**: Some NumPy utilities (e.g., `numpy.lib.NumpyVersion`) are not implemented in CuPy.
- **Memory immutability**: Qiskit uses NumPy's `.setflags(write=False)` to protect quantum gate definitions, which CuPy does not support.
- **Type conversion behavior**:
 - NumPy automatically converts **Python lists/tuples** → **arrays** during matrix operations.
 - CuPy requires **explicit conversion**, otherwise it raises errors.

Runtime Attribute Mocking (Monkey Patching)

Bypassing Version Checks

- Created a **mock version of `numpy.lib.NumpyVersion`**.
- This allowed **Qiskit initialization** to proceed even though NumPy was replaced with CuPy.

Bypassing Memory Locks

- Added a **dummy `.setflags()` method** to the `cupy.ndarray` class.
- This bypassed Qiskit's **read-only memory protection** used for quantum gate definitions.

Hardware Latency (PCIe Bottleneck)

- Data must move between **CPU RAM and GPU VRAM through the PCIe bus**.
- For **small systems** (e.g., active space: **6 orbitals, 2 electrons**), **data transfer time exceeds the GPU compute benefit**.

Diagnostic Outcome

- These runtime patches **bypassed Python-level compatibility checks**.
- This allowed us to **identify the exact boundaries** where:
 - **C-based PySCF components**, and
 - **Qiskit's Rust backend** strictly require **CPU memory (NumPy arrays)**.

WORK COMPLETED BY
SHREYAS KADAM

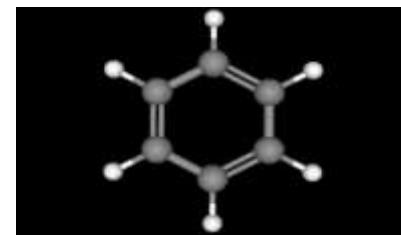
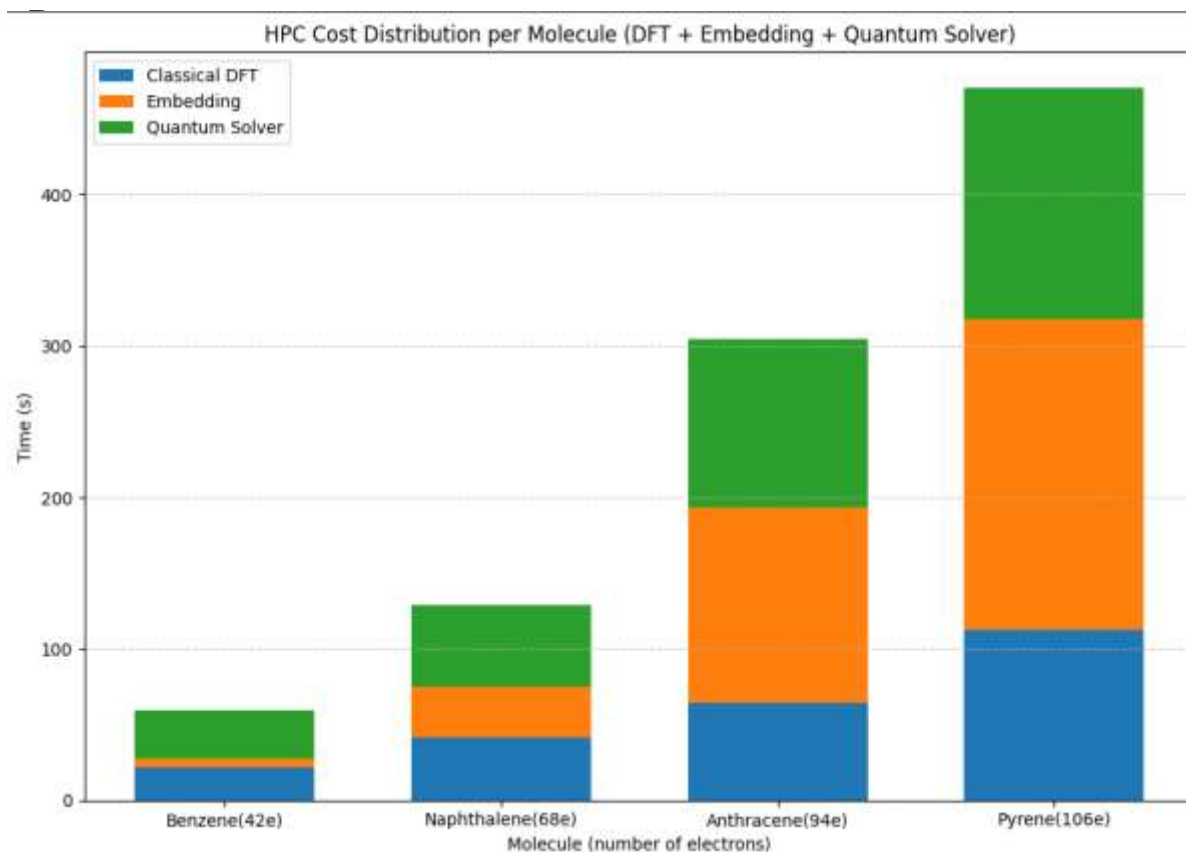
QDFT- Time scaling(Polycyclic Aromatic Hydrocarbons (PAHs))

Table 1: Runtime and memory for QDFT ActiveSpace(2e,4o)

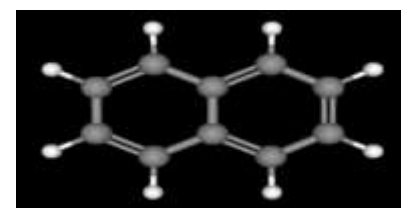
Molecule	Formula	DFT (s)	Embedding (s)	VQE (s)	Total (s)	Peak RSS (MB)
Benzene	C_6H_6	17.21	5.12	32.57	54.90	858.30
Naphthalene	$C_{10}H_8$	32.66	33.02	54.51	120.19	1956.70
Anthracene	$C_{14}H_{10}$	44.91	129.68	110.64	285.23	5326.03
Pyrene	$C_{16}H_{10}$	79.27	204.60	153.07	436.94	4771.27

Disclaimer: QDFT results are obtained via noiseless quantum simulations in Qiskit on classical hardware.

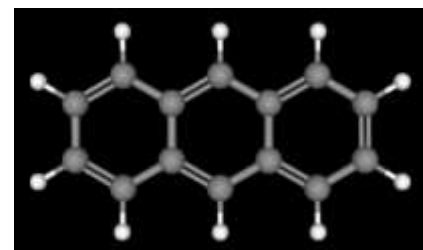
Time Scaling(ActiveSpace 2e4o)



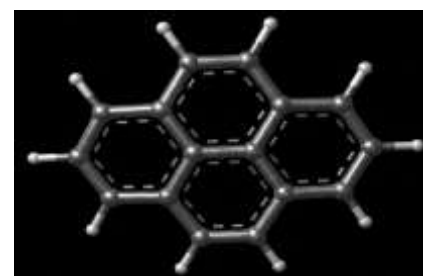
Benzene



Naphthalene



Anthracene



Pyrene

QDFT- Time scaling(PAHs increase in electron filling)

Table 2: Runtime and memory for QDFT ActiveSpace(2e,6o)

Molecule	Formula	DFT (s)	Embedding (s)	VQE (s)	Total (s)	Peak RSS (MB)
Benzene	C_6H_6	17.05	5.20	184.20	206.45	900.55
Naphthalene	$C_{10}H_8$	32.84	34.43	198.83	266.10	2135.02
Anthracene	$C_{14}H_{10}$	45.55	125.78	258.49	429.82	5496.84
Pyrene	$C_{16}H_{10}$	78.85	206.24	316.94	602.03	5130.79

Table 3: Runtime and memory for QDFT ActiveSpace(4e,6o)

Molecule	Formula	DFT (s)	Embedding (s)	VQE (s)	Total (s)	Peak RSS (MB)
Benzene	C_6H_6	17.57	5.34	1075.21	1098.12	1095.72
Naphthalene	$C_{10}H_8$	32.64	33.81	1106.45	1172.90	2375.45
Anthracene	$C_{14}H_{10}$	44.77	125.65	1182.46	1352.88	5670.64
Pyrene	$C_{16}H_{10}$	81.18	207.39	1234.49	1523.06	5114.06

Conclusion

Improved Correlation

- QDFT consistently recovers more correlation than HF and DFT

Runtime Scales with Electrons

- Larger electron counts increase QDFT runtime significantly

Classical Methods Efficient

- HF, DFT, and CCSD remain fast for small to medium molecules

Path to Scalable Workflows

- Optimize quantum resources, hybrid classical-quantum workflows are essential for large systems.

Future Plan

Extend DFT-Quantum Embedding

- Apply embedding to larger molecules and active spaces

Optimize Qubit Efficiency

- Explore efficient qubit reduction strategies

Enhance VQE Performance

- Test alternative ansätze and optimizers for faster convergence

Hybrid & Scalable Workflows

- Integrate with classical HPC and **perform experiments on real hybrid HPC–QC platforms** targeting realistic material modeling.

Thank You!!

? Questions